Key management & crypto protocols

Lecture plan

- Intro and attacker modus
- Software and web app security
- System security
- **Crypto**
- Network security
- Reactive security
- Outro

Agenda

- **Last time:**
  - Crypto building blocks
- **Today:**
  - Crypto protocols

Remember Kerckhoffs

Many keys to protect

- Master key
- Session key
- Signature key
- Data encryption key
- Key encryption key
- ...

Protect during entire lifecycle

- Generation
- Exchange
- Storage/backup
- Use
- Expiration
- Revocation
- Destruction
- (Repeat)

Key exchange options include

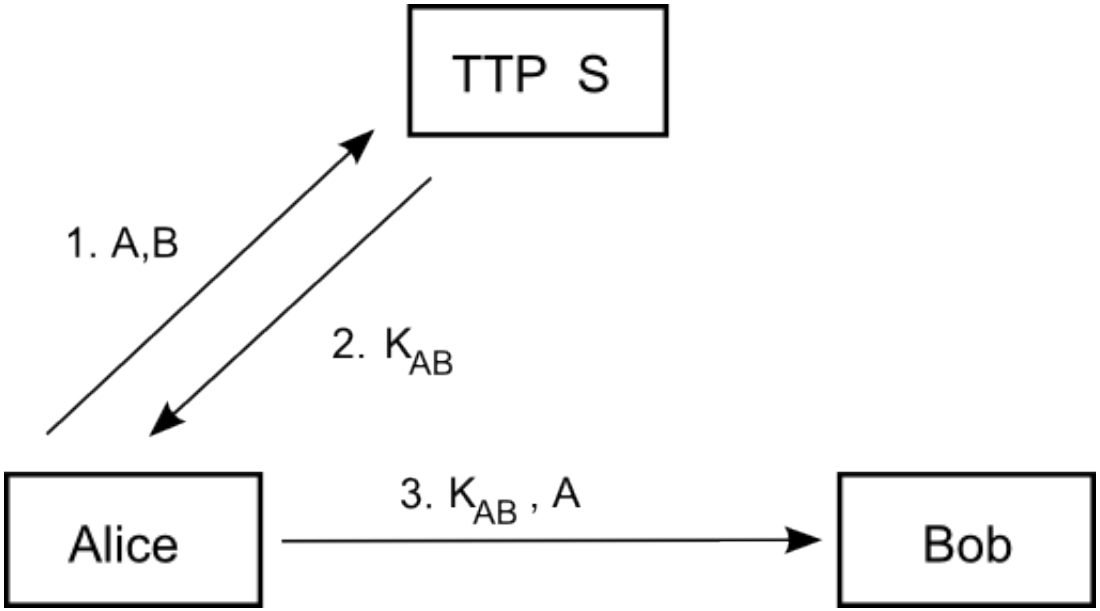
- Pre-distribution
  - Generated and distributed “ahead of time” e.g. physically
- Distribution
  - Generated by a trusted third party (TTP) and sent to all parties
- Agreement
  - Generated by all parties working together
- ASYM: Is e really yours?

Key distribution
(Trusted third party active participant)

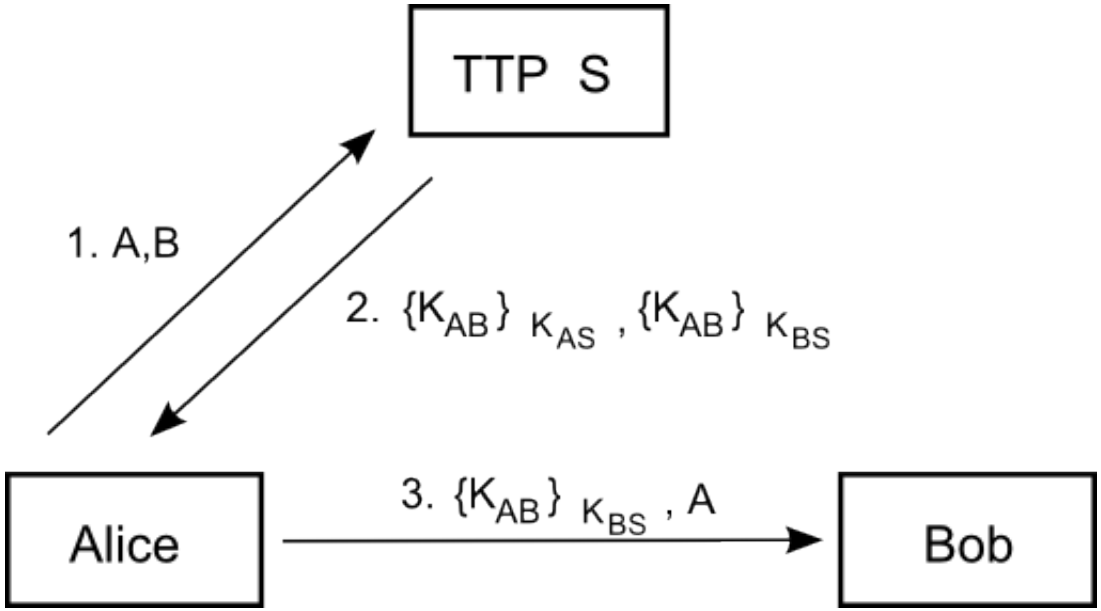
Developing a (session) key distribution scheme

- Situation:
  - A and B want to exchange keys remotely
  - Both A and B share a key (K_AS, K_BS) with a trusted third party, S
- Goals:
  - At the end, we want to achieve:
  - A and B know a new key K_AB
  - No one but A, B, and possibly S knows K_AB
  - A and B know that K_AB is newly generated

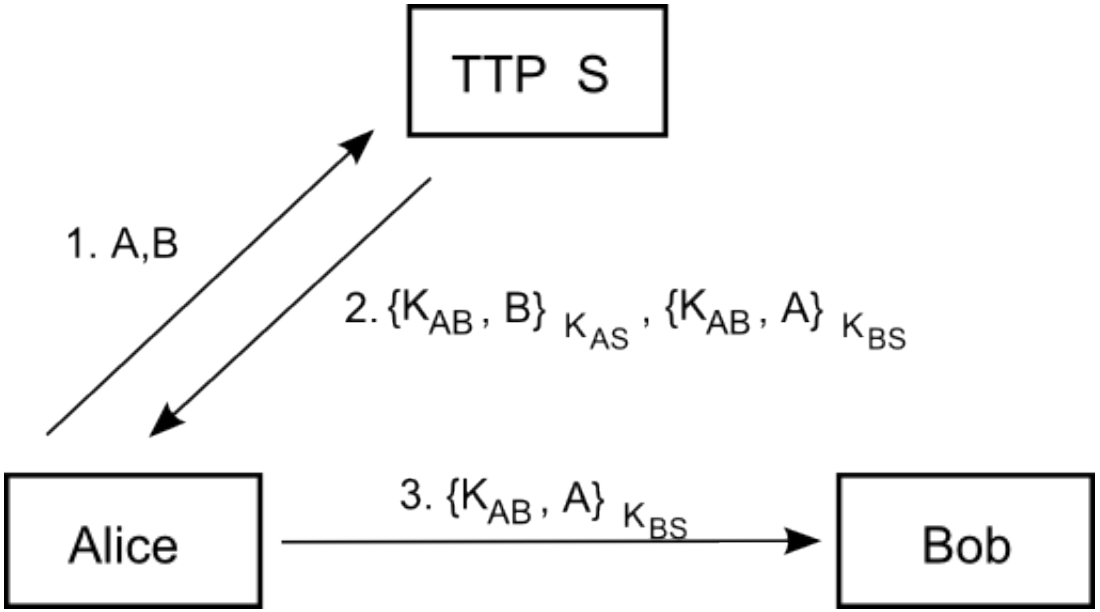
First attempt



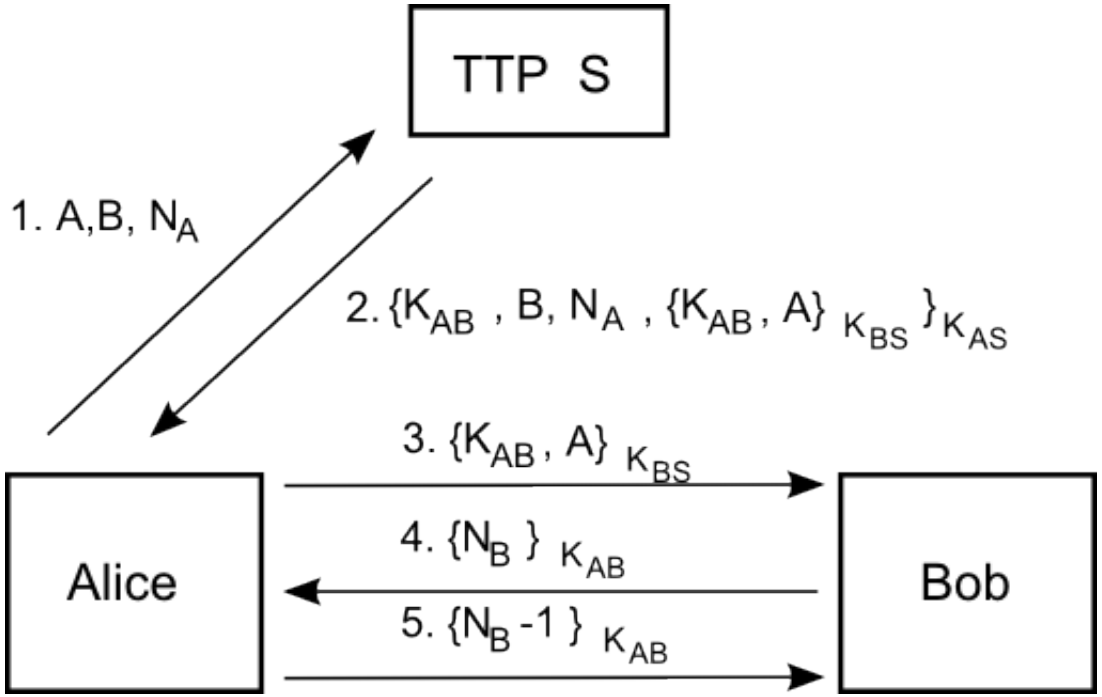
Second attempt



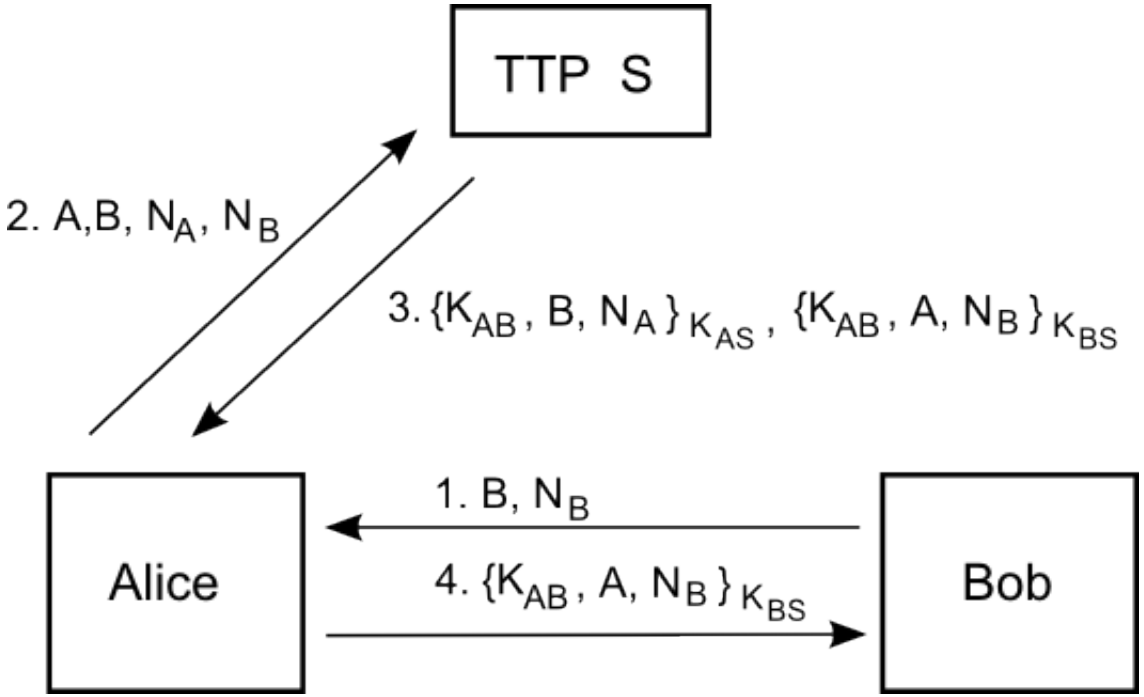
Third attempt



Needham-Schroeder

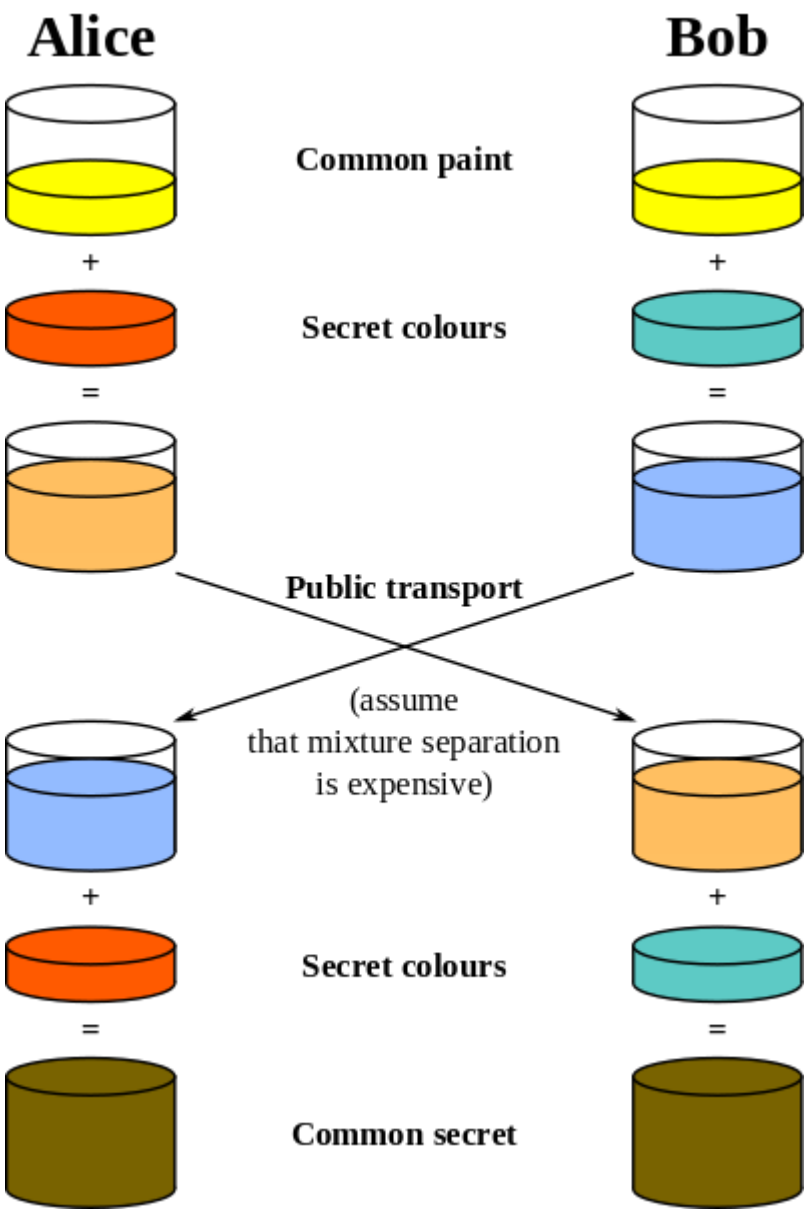


Denning-Sacco



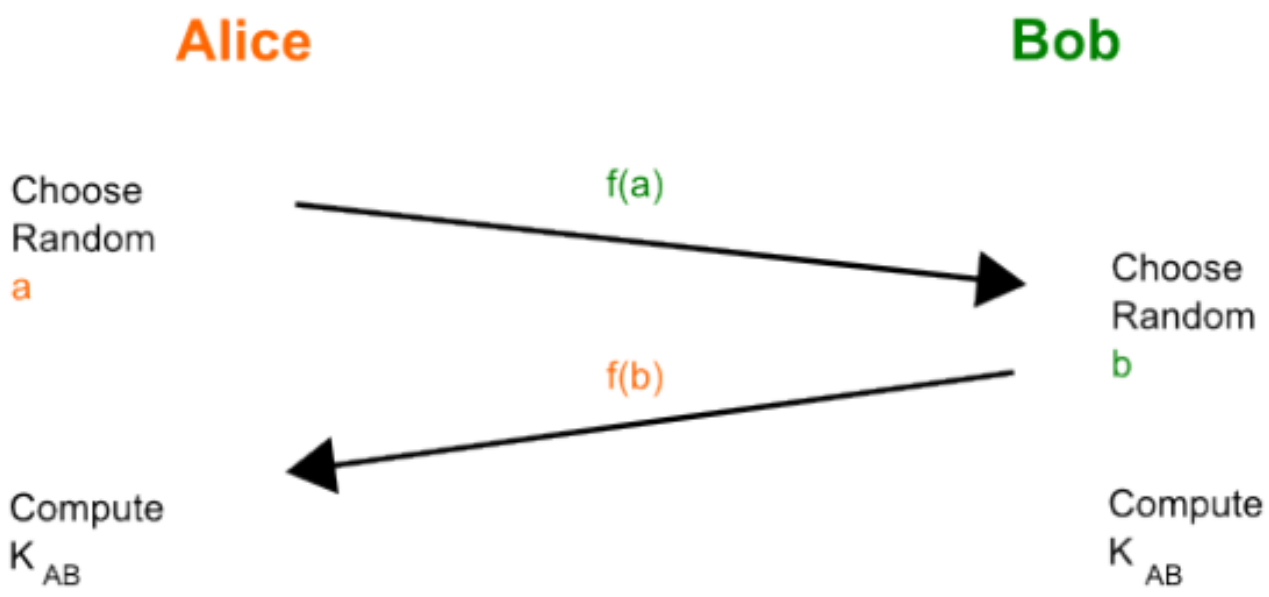
Key agreement
(Parties working together agreeing on the key)

Diffie-Hellman idea



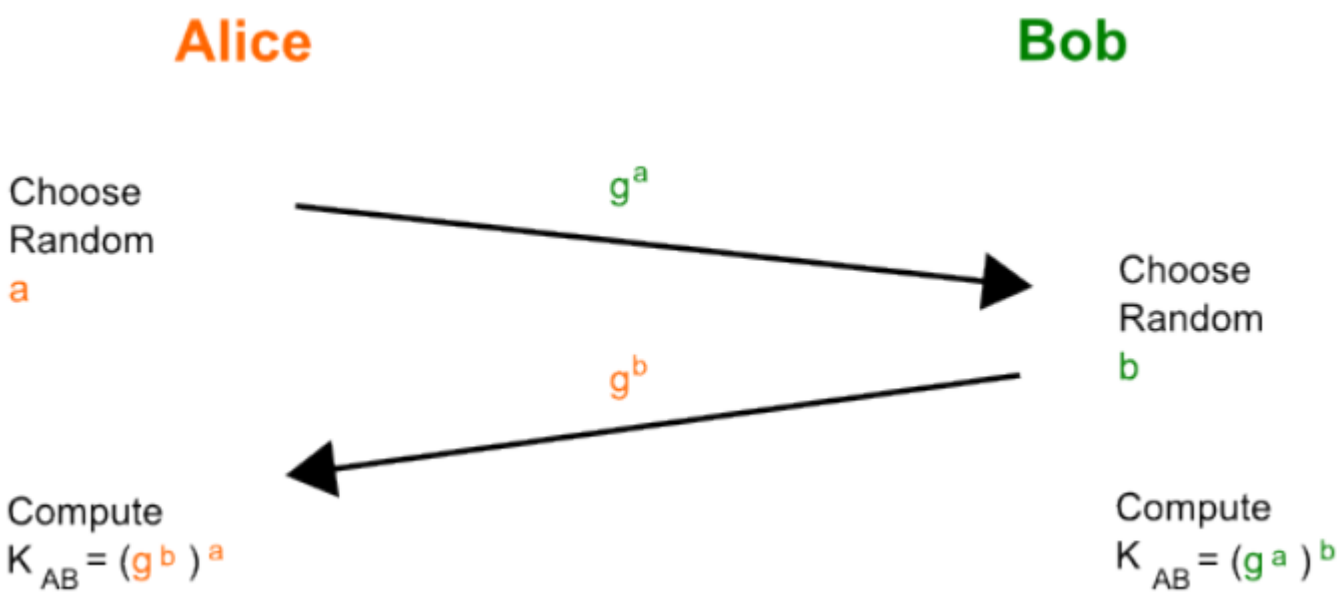
Non-paint version

- Instead of paint, we need a function f(x)



Solution by Diffie-Hellman, 1976

- Modular exponentiation:
  - Let p prime and g<p be fixed and public
  - $f(x) = g^x \bmod p$



Security of DH

- Given $A=g^a$, find x so $g^x=g^a$
  - Discrete logarithm problem

- Given $A=g^a$ and $B=g^b$, find g^{ab}
  - Computational Diffie-Hellman assumption

PKI
(Is *e* really yours?)

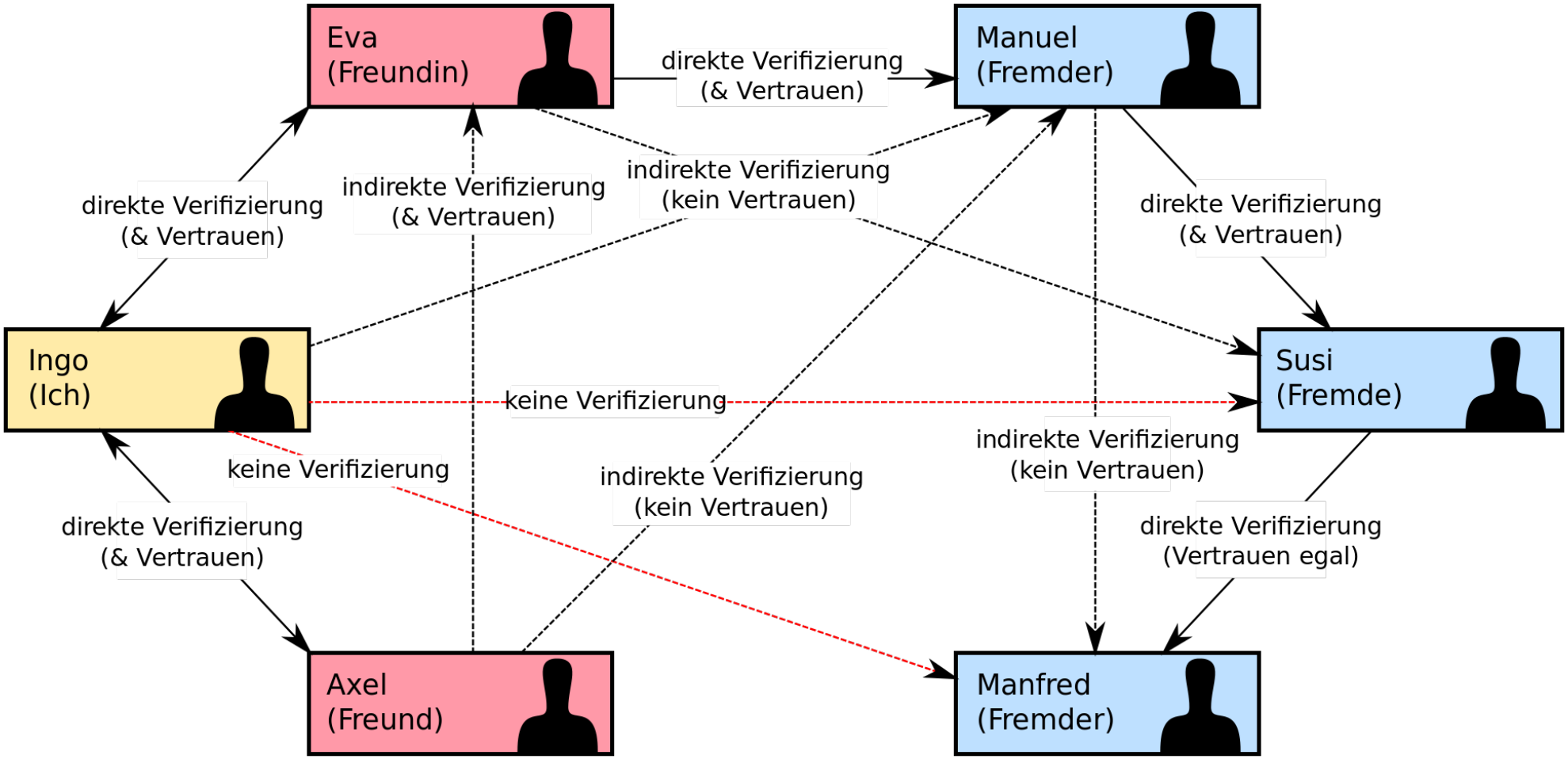
PKI definition

- A public key infrastructure (PKI) is <u>a system for the creation, storage, and distribution of digital certificates</u> which are used to verify that a particular public key belongs to a certain entity

What's a certificate?

- Certificate: An electronic document used to prove ownership of a public key.
- X.509 format for certificates include:
  - Serial number – unique identification of certificate
  - Valid-From/To – lifespan of the certificate
  - Subject – the entity/person/machine/etc. identified
  - Public key – the entity's public key
  - Key usage – purpose of the public key (encryption, signing, etc.)
  - Issuer – entity that verified above and issued the certificate
  - Signature – the actual signature of the issuer
  - Signature algorithm – how it was made

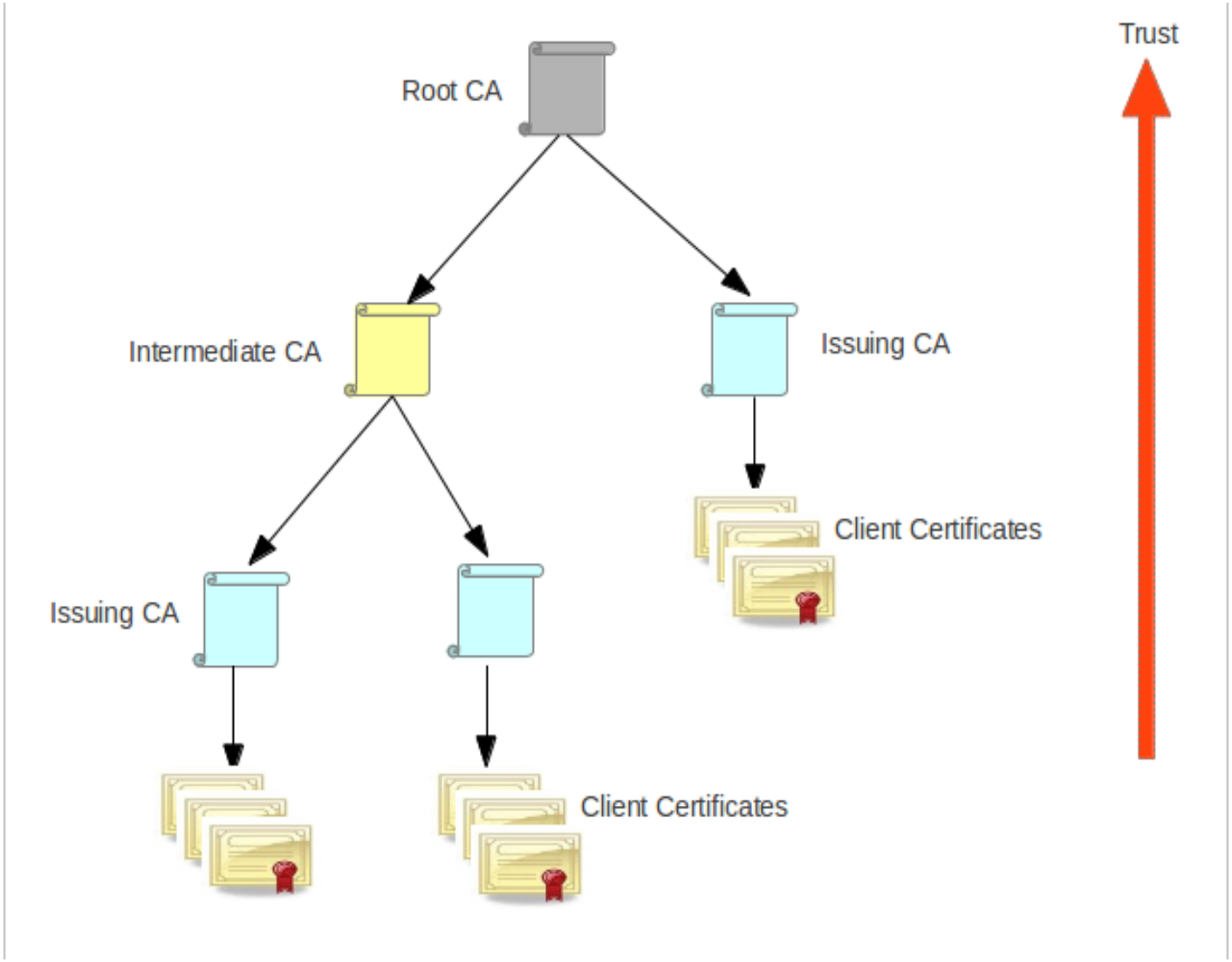
Web of trust example



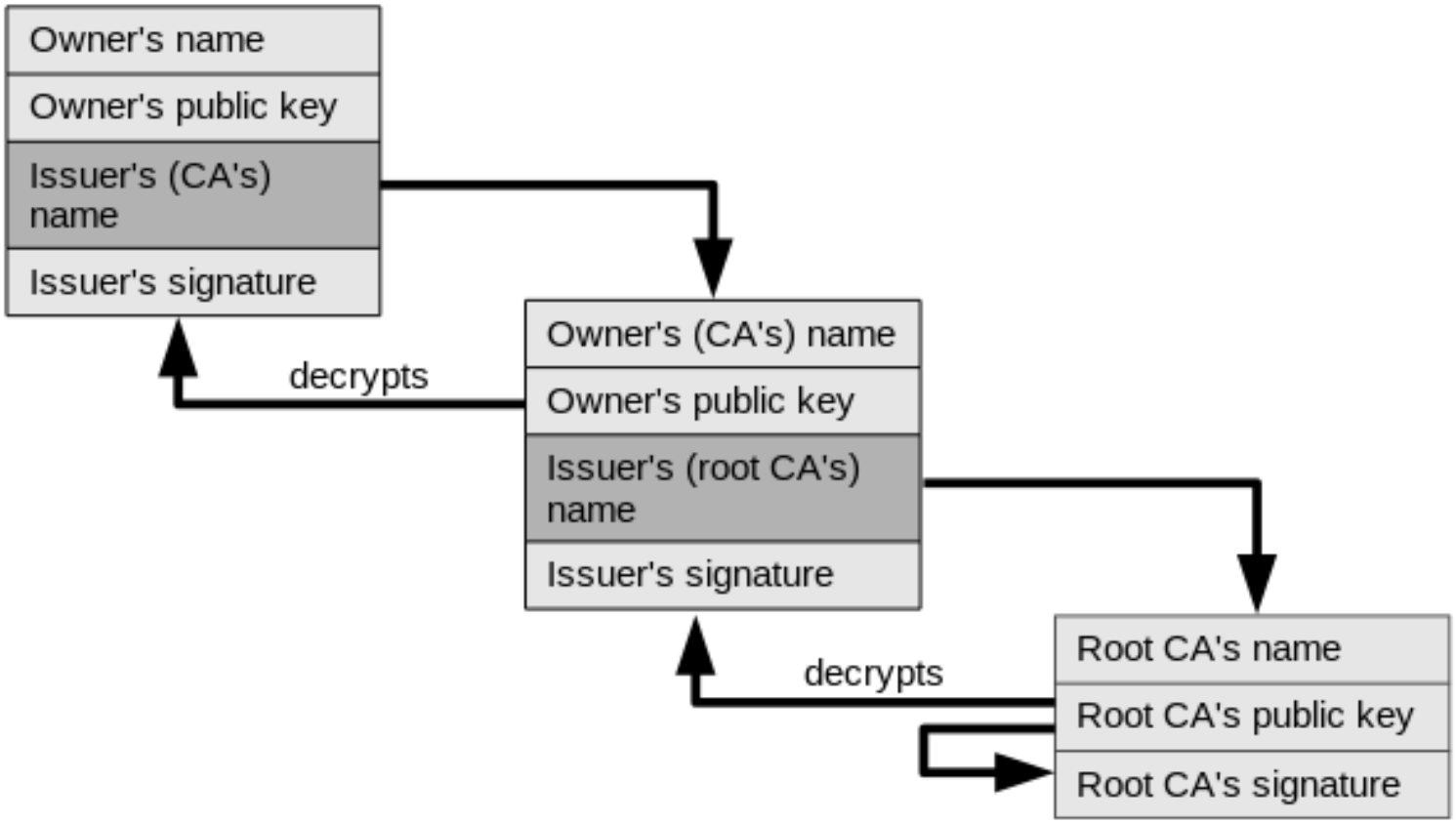
Web of trust

- Decentralised model where certificates are signed by other users who, by that act, endorse the association of that public key with the entity listed in the certificate

The CA model

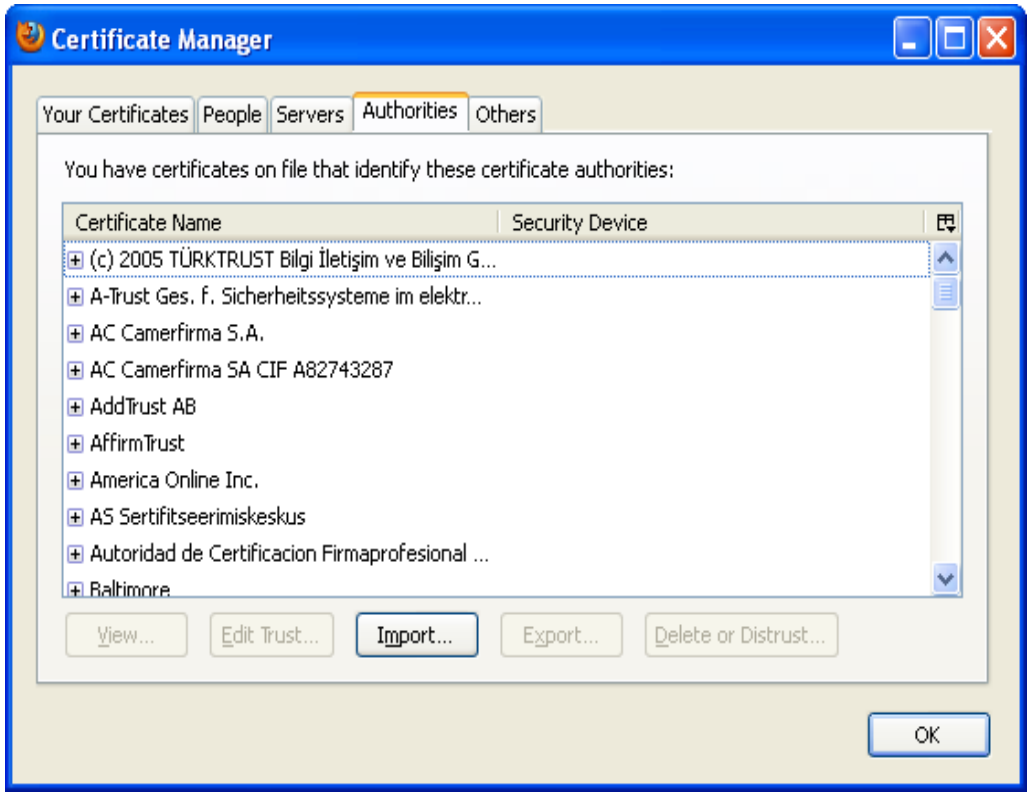

Trust
Root CA
Intermediate CA
Issuing CA
Client Certificates
Issuing CA
Client Certificates

Chain of trust



Trust in web browsers

- Browsers come pre-configured with a set of root CAs. Do you trust all these CAs (to authenticate properly, to avoid/inform of breaches)?



Certificate providers

A W3Techs survey from April 2016 shows:[11]

Rank	Issuer	Usage	Market share
1	Comodo	8.1%	40.6%
2	Symantec	5.2%	26.0%
3	GoDaddy	2.4%	11.8%
4	GlobalSign	1.9%	9.7%
5	IdenTrust	0.7%	3.5%
6	DigiCert	0.6%	3.0%
7	StartCom	0.4%	2.1%
8	Entrust	0.1%	0.7%
9	Trustwave	0.1%	0.5%
10	Verizon	0.1%	0.5%

Let's encrypt

- Let's Encrypt is a nonprofit certificate authority that launched on April 12, 2016 that provides free X.509 certificates

Revocation

- Certificate revocation list (CRL):
  - A list of (serial numbers for) certificates that have been revoked, and therefore, entities presenting those (revoked) certificates should no longer be trusted
- Online Certificate Status Protocol (OCSP):
  - Protocol used for obtaining the revocation status of an X.509 digital certificate

Subvertion of CAs

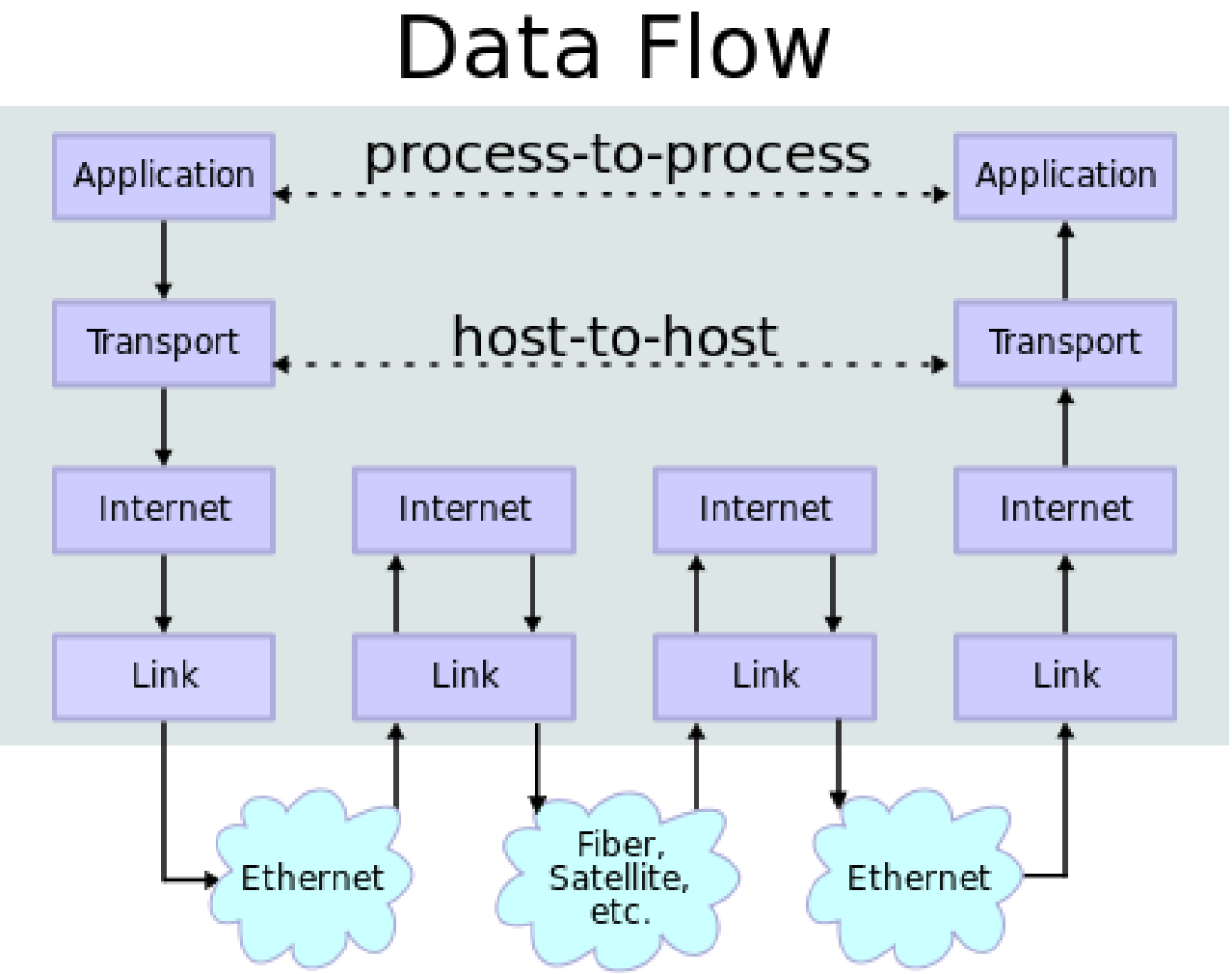
- If (root) CA is subverted, security of entire chain lost
- How to subvert:
  - Claim to be someone else (fraudulent Microsoft certs issued by VeriSign, 2001)
  - Breach CA and issue fraudulent certs (Comodo and DigiNotar, 2011)
- Study by EFF (2011): In at least 248 cases, a CA chose to indicate that it had been compromised as a reason for revoking a certificate

Remember Apple iOS Goto Fail

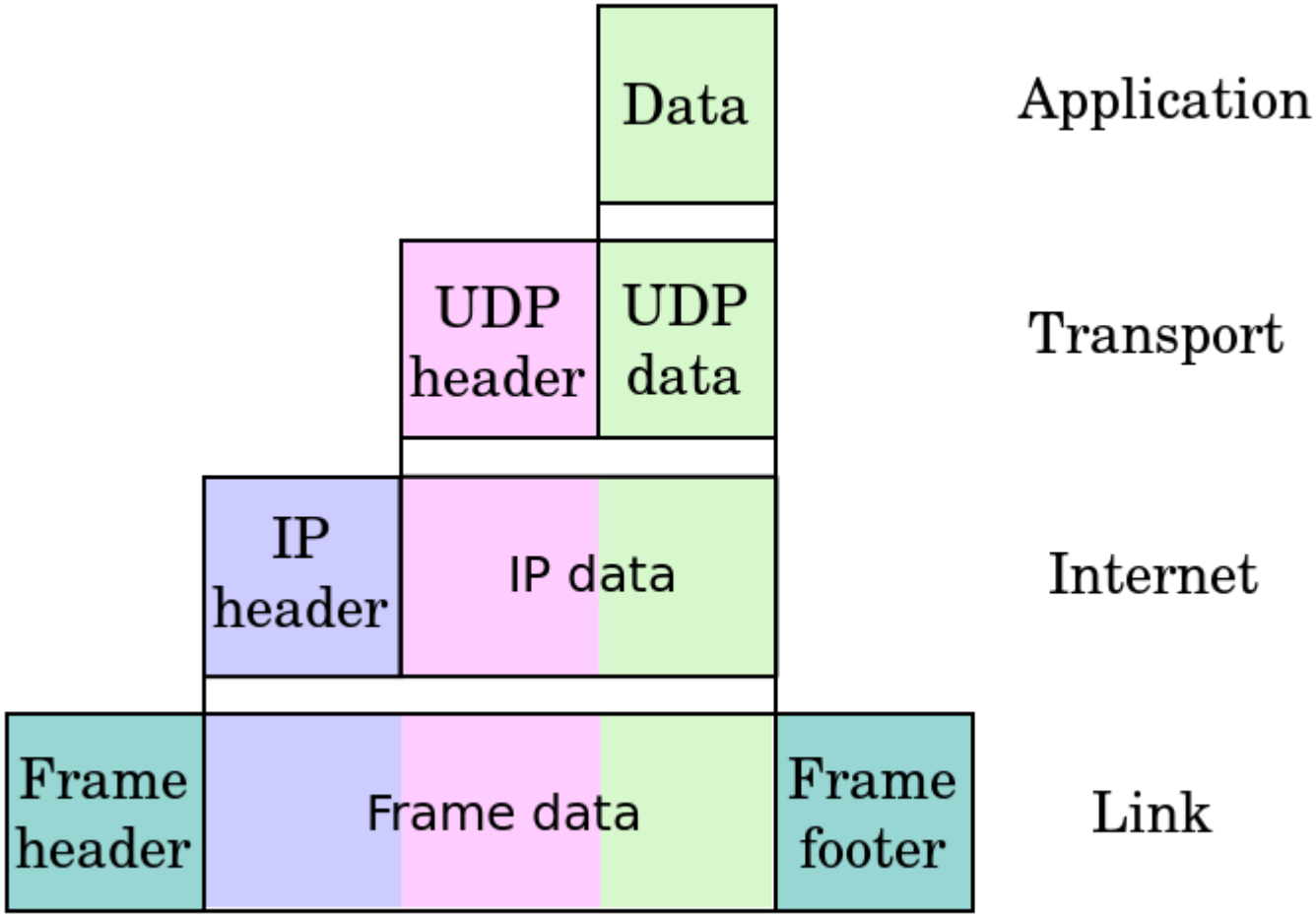
```
static OSStatus
SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa, SSLBuffer signedParams,
                                 uint8_t *signature, UInt16 signatureLen)
{
    OSStatus        err;
    ...

    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;
    ...

fail:
    SSLFreeBuffer(&signedHashes);
    SSLFreeBuffer(&hashCtx);
    return err;
}
```

Crypto protocols

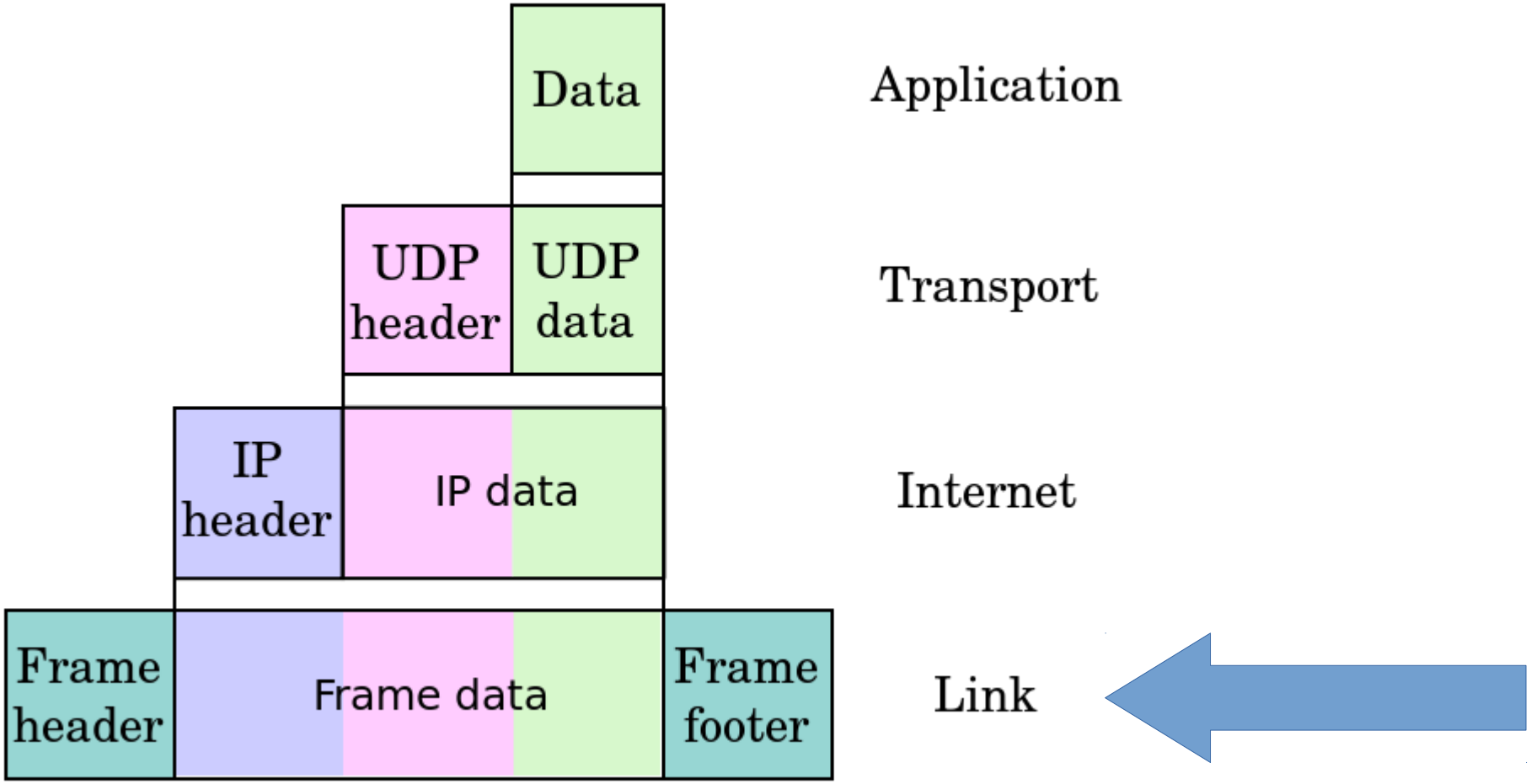
Where to encrypt?



Where to encrypt?



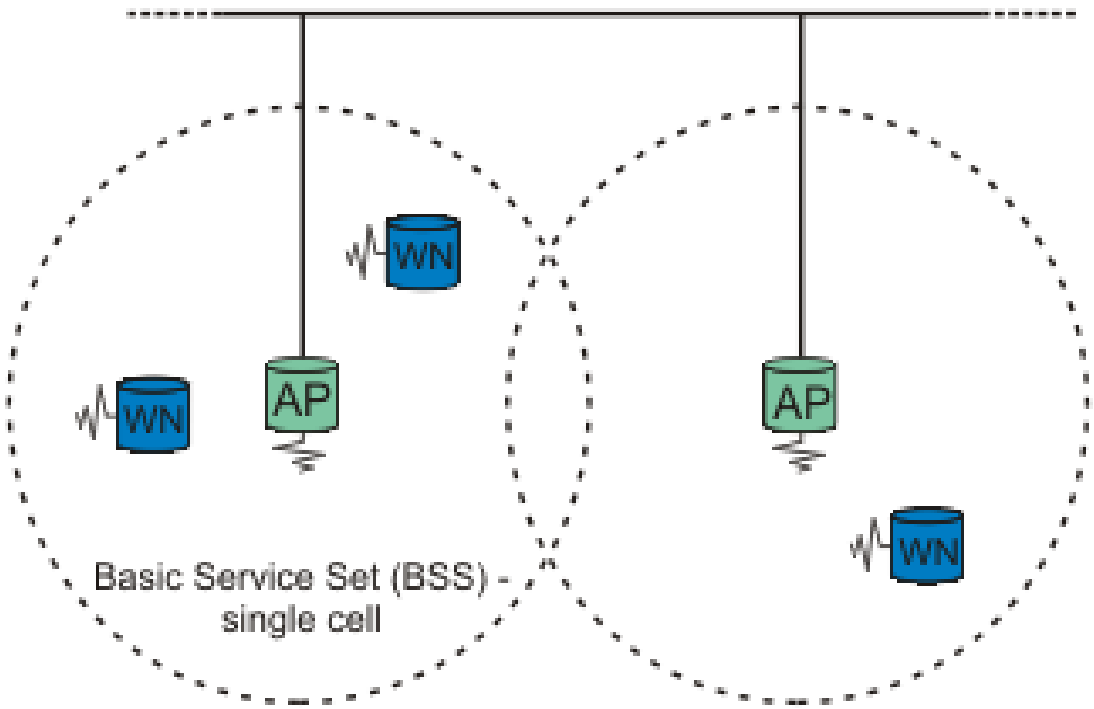
Where to encrypt?



Wifi

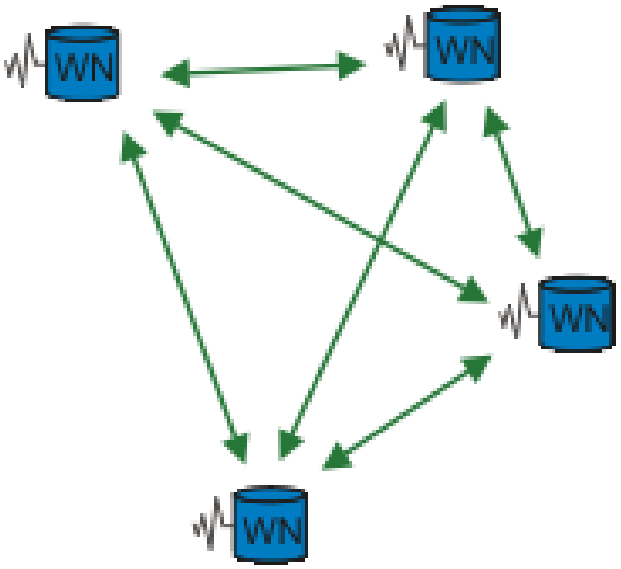
IEEE 802.11

Infrastructure Mode:



Extended Service Set (ESS) - multiple cells

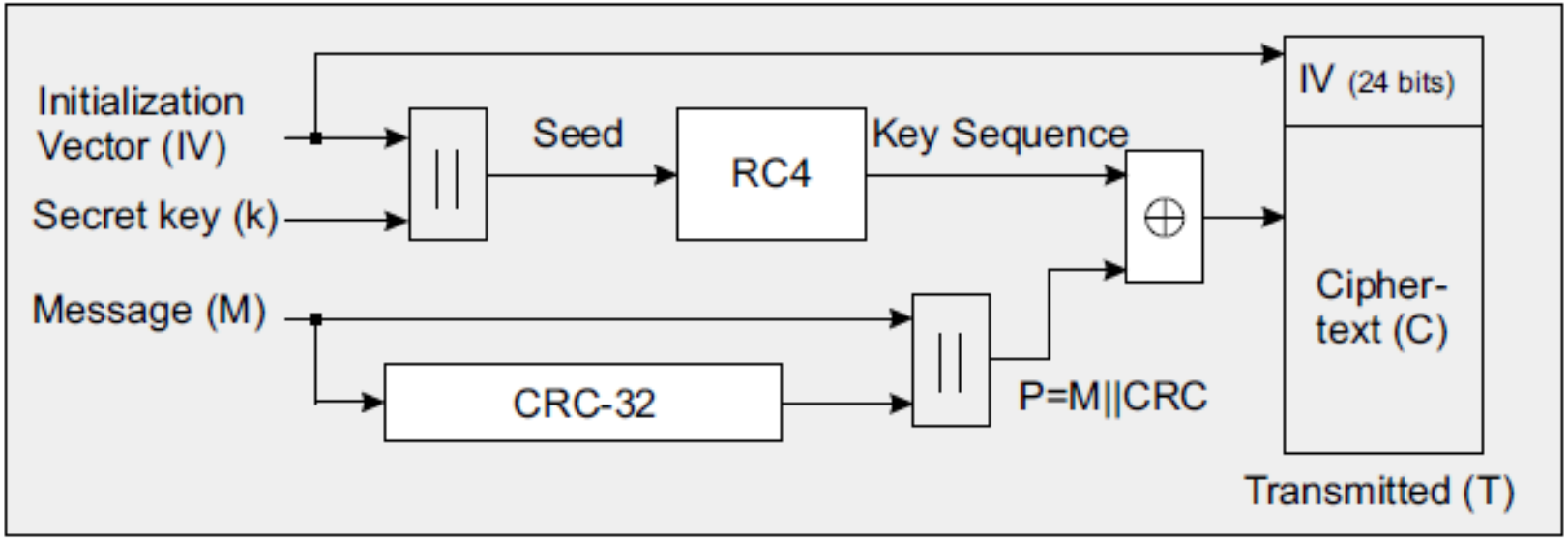
IEEE Ad-hoc Mode:

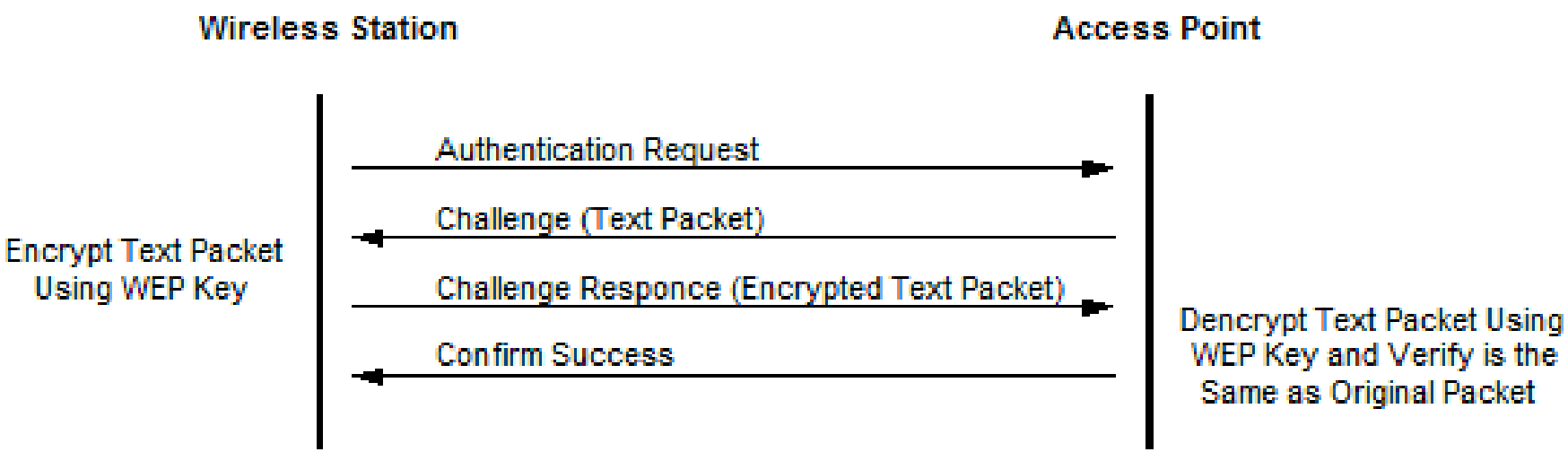


Security in IEEE 802.11

Protocol	Encryption	Authentication	Key Management
WEP	• RC4 with 40 bit key/28 bit hash • Static keys	• Pre Shared keys • Open System (SSID)	• Manual key rotation, i.e., no key management
WPA	• TKIP with 128 bit key (over RC4) • Constant key rotation	• 802.1x with EAP and RADIUS • Pre-shared key	• Per packet key rotation
802.11i	• TKIP with 128 bit key (over RC4) • AES-CCMP • Constant key rotation	• 802.1x with EAP and RADIUS • Pre-shared key	• Per packet key rotation (TKIP) • Per session key rotation (AES-CCMP)

WEP


Initialization Vector (IV)
Secret key (k)
Message (M)
||
Seed
RC4
Key Sequence
⊕
IV (24 bits)
Cipher-text (C)
||
CRC-32
P=M||CRC
Transmitted (T)


Wireless Station
Access Point
Authentication Request
Challenge (Text Packet)
Challenge Responce (Encrypted Text Packet)
Confirm Success
Encrypt Text Packet Using WEP Key
Dencrypt Text Packet Using WEP Key and Verify is the Same as Original Packet

WEP

- Encryption
  - RC4, 40-bit key, 24-bit IV
- Authentication
  - Pre-shared keys
- Key management
  - None

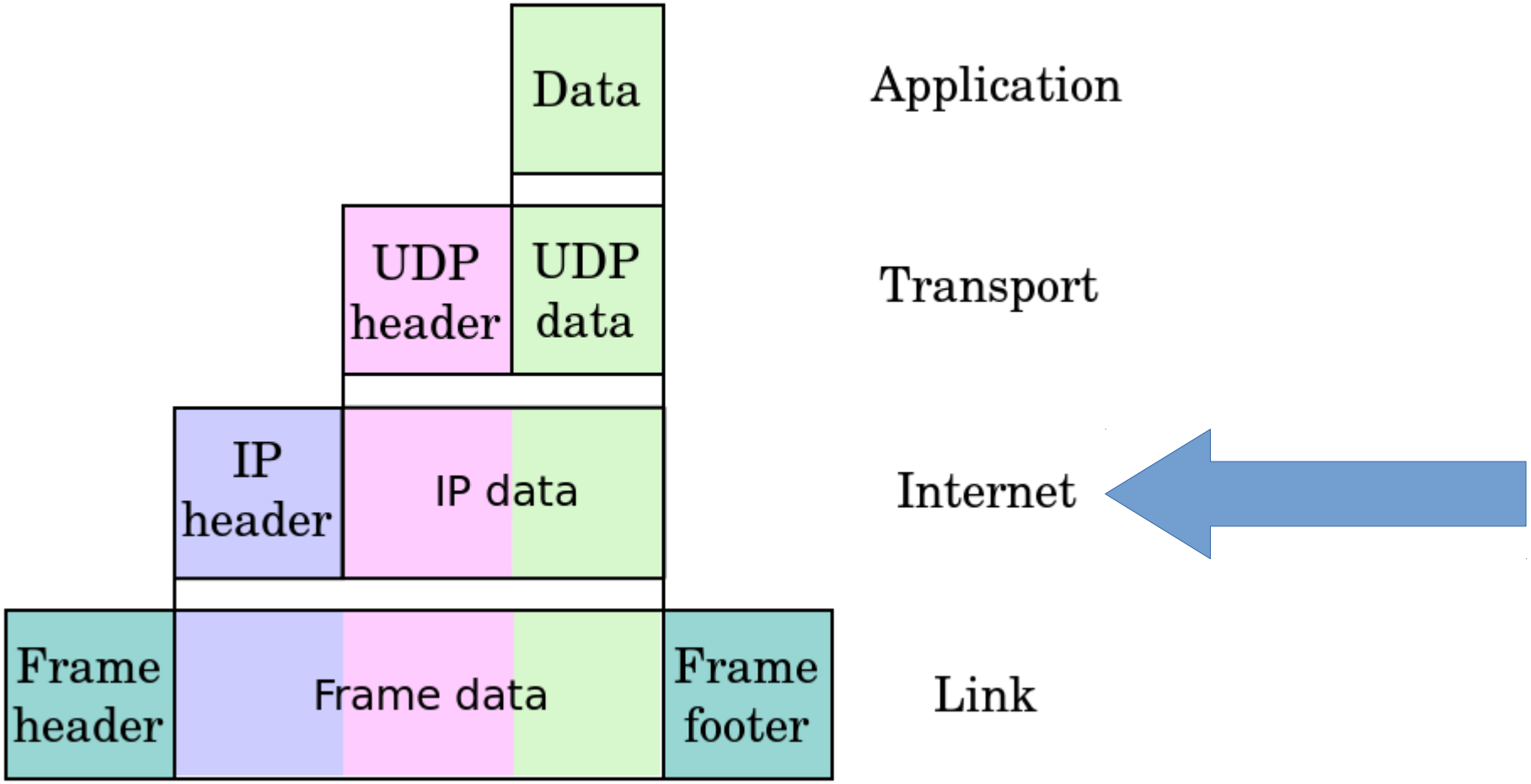
The final nail in WEP's coffin

- The FMS-attack (Fluhrer, Mantin and Shamir, 2001)
  - Finds k when "weak" IVs used
  - Originally required 4m pkcts, down to <0.5m (KoreK, 2004)
- The Klein-attack (Klein, 2006)
  - Finds k regardless of weak or non-weak Ivs
  - Requires <0.1m pckts

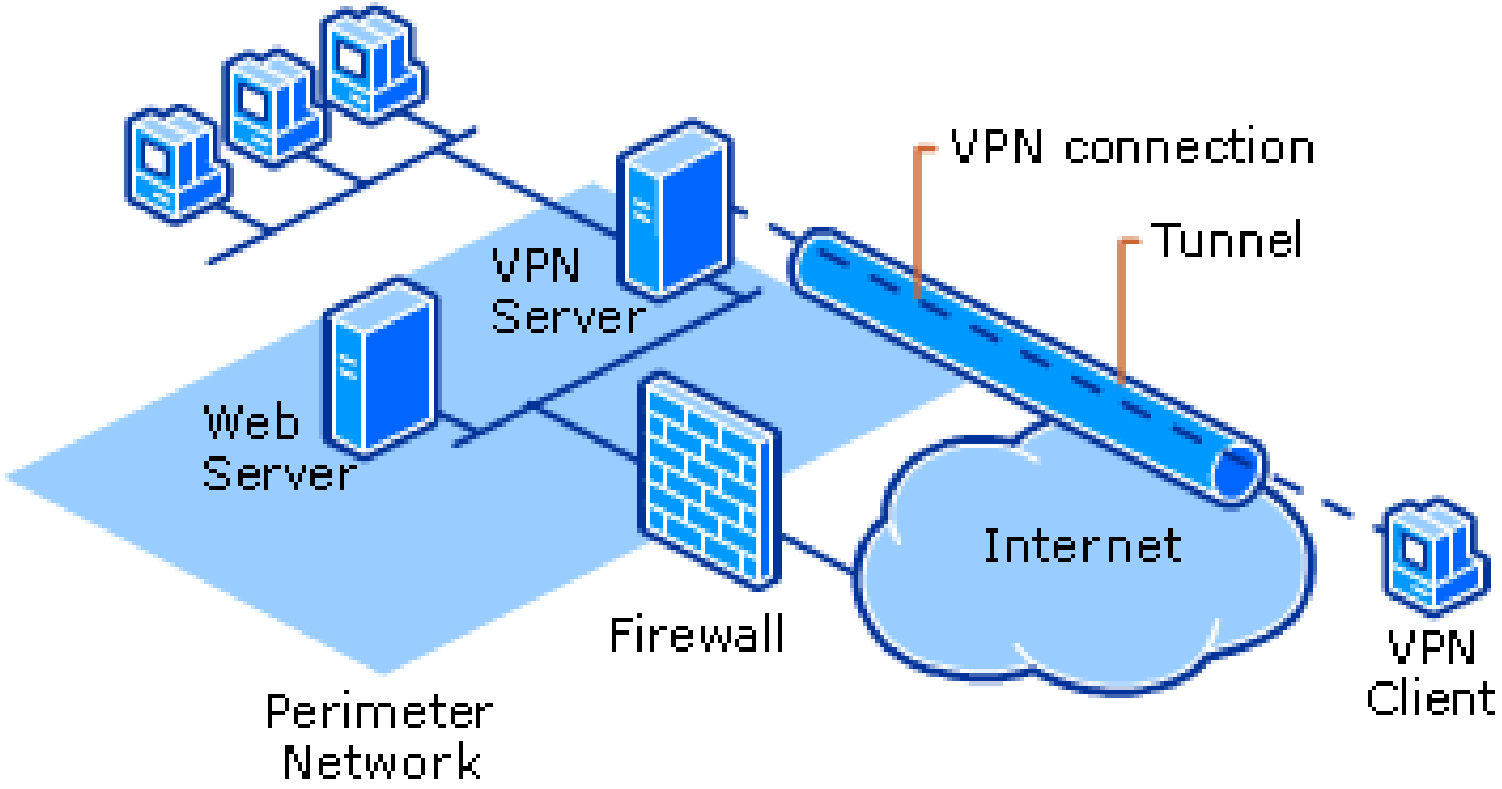
WPA / WPA2

- 4-way mutual authentication handshake
  - Personal: Like WEP based on pre-shared key
  - Enterprise: Credentials, certificates, etc. via IEEE 802.1X to a back-end authentication server
- Key management with various subkeys
- Stronger confidentiality and integrity protection (TKIP/CCMP)
- Replay protection

Where to encrypt?

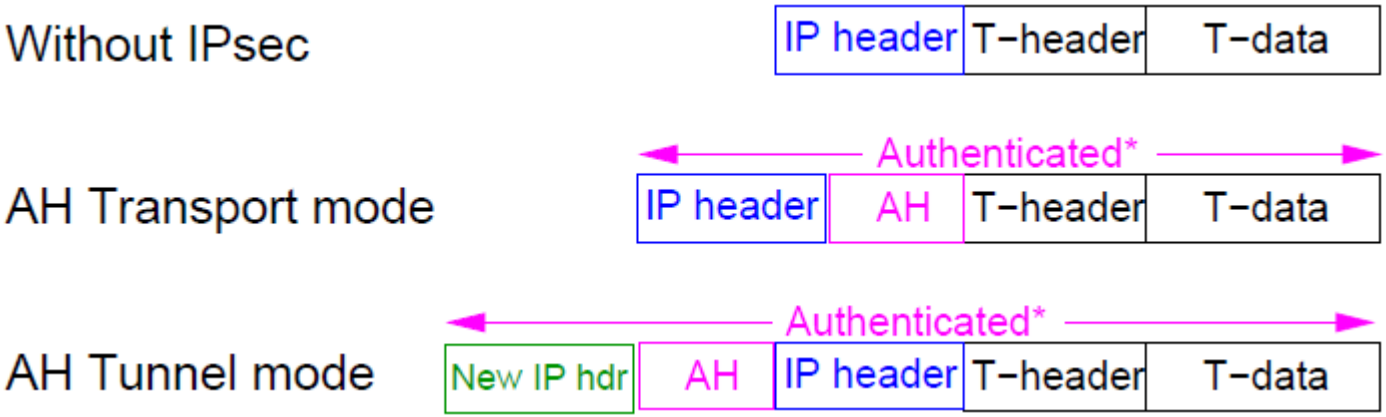


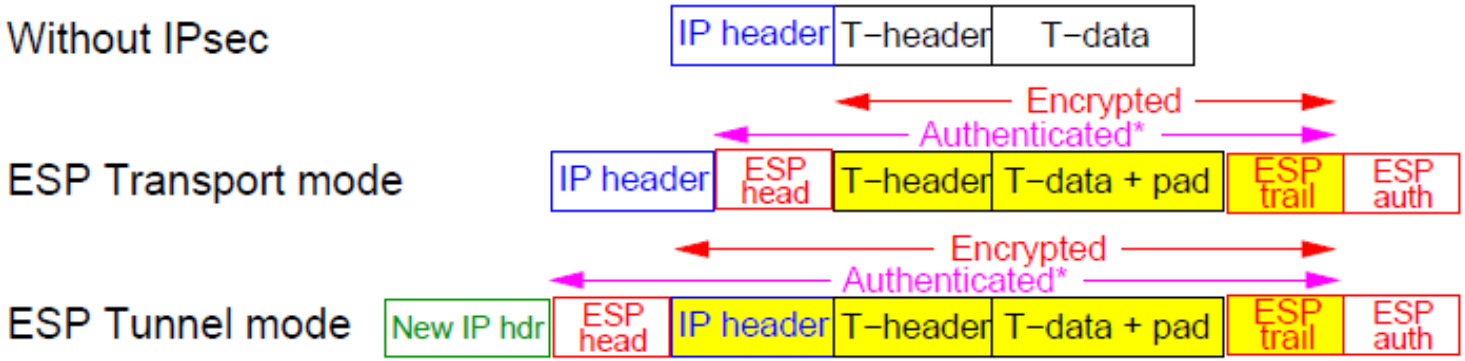
Virtual Private Network (VPN)



IPsec

IPsec AH, ESP


Without IPsec
IP header
T−header
T−data
Authenticated*
AH Transport mode
IP header
AH
T−header
T−data
Authenticated*
AH Tunnel mode
New IP hdr
AH
IP header
T−header
T−data


Without IPsec
IP header
T−header
T−data
Encrypted
Authenticated*
ESP Transport mode
IP header
ESP head
T−header
T−data + pad
ESP trail
ESP auth
Encrypted
Authenticated*
ESP Tunnel mode
New IP hdr
ESP head
IP header
T−header
T−data + pad
ESP trail
ESP auth

Internet Protocol Security (IPsec)

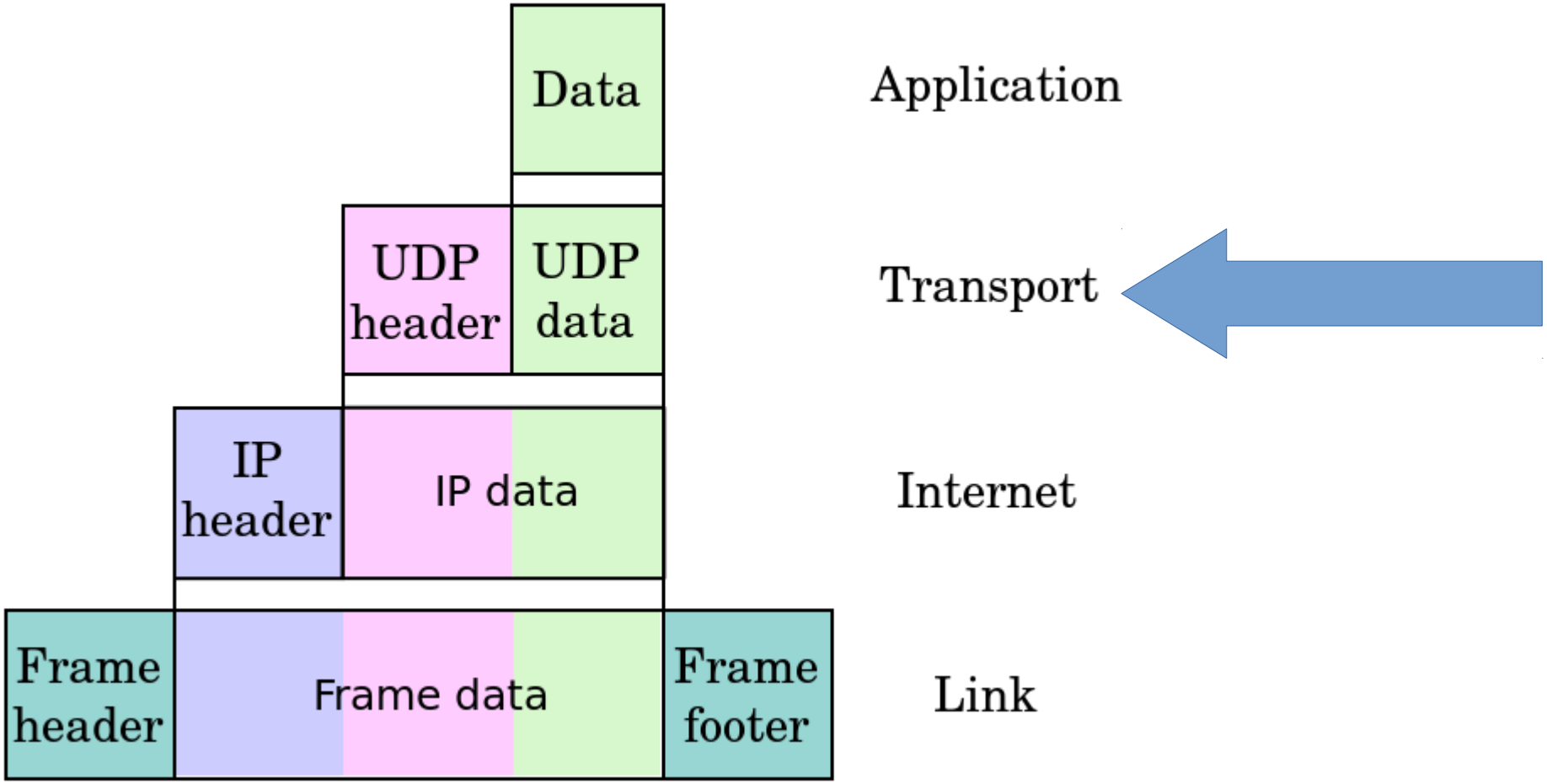
- RFC2401: Overview of security architecture
- RFC2409: Internet Key Exchange (IKE)
- RFC4302: IP Authentication Header (AH)
- RFC4303: IP Encapsulating Security Payload (ESP)
- Part of IPv6, add-on for IPv4

IPsec IKE

- IKE sets up a security association (SA) - one SA for each direction of data flow
- X.509 certificates for authentication
- Diffie–Hellman key exchange

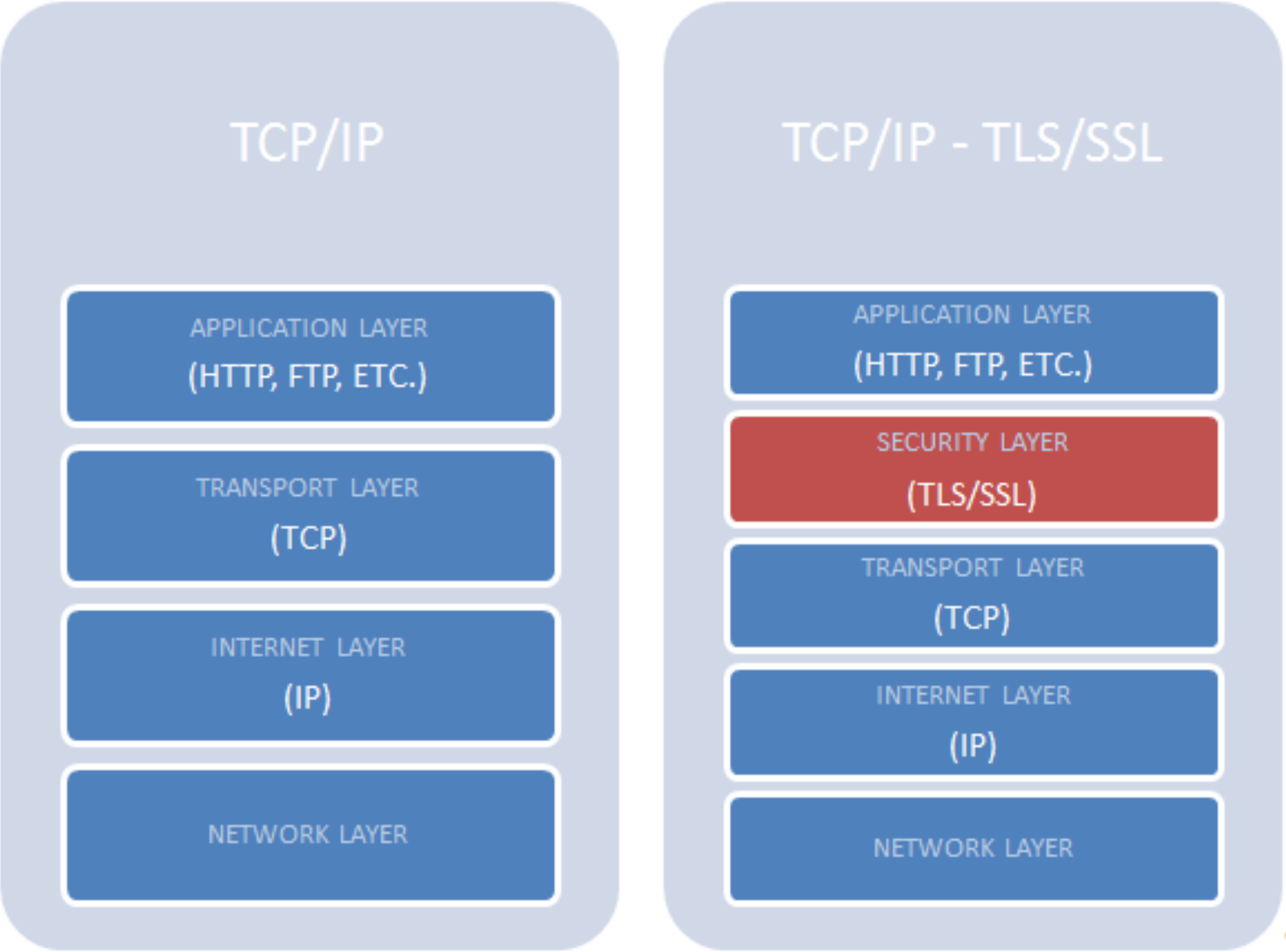
```
vyatta@r1:~$ show vpn ipsec sa detail
Conn Name:            peer-192.168.22.234-tunnel-1
State:                up
Peer:                 192.168.22.234
Direction:            in
Source Net:           192.168.210.192/29
Dest Net:             192.168.110.2/32
SPI:                  7b240247
Encryption:           aes128
Hash:                 sha1
PFS Group:            Phase1
DH Group:             5
```

Where to encrypt?

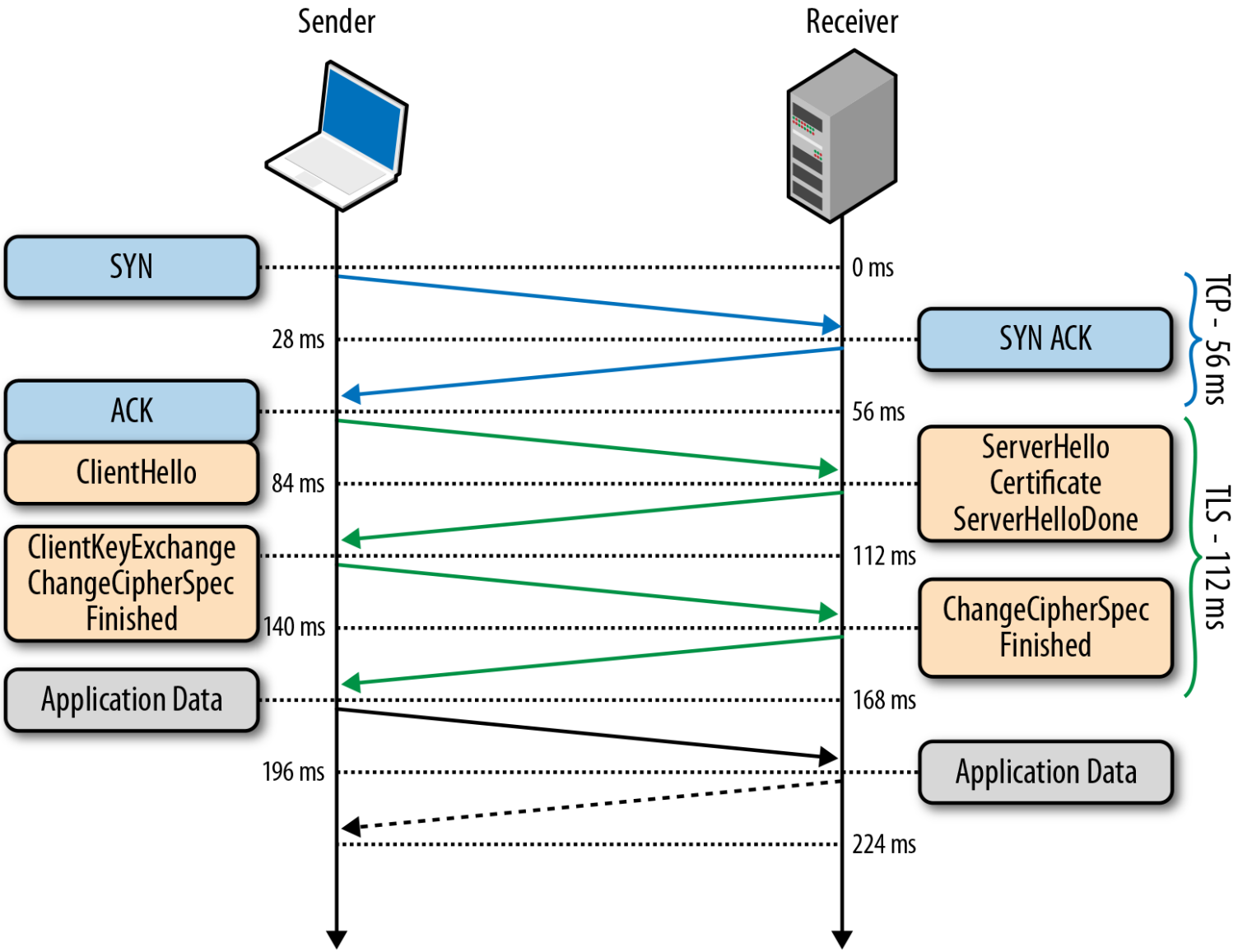


TLS/SSL

TLS/SSL


TCP/IP
APPLICATION LAYER
(HTTP, FTP, ETC.)
TRANSPORT LAYER
(TCP)
INTERNET LAYER
(IP)
NETWORK LAYER
TCP/IP - TLS/SSL
APPLICATION LAYER
(HTTP, FTP, ETC.)
SECURITY LAYER
(TLS/SSL)
TRANSPORT LAYER
(TCP)
INTERNET LAYER
(IP)
NETWORK LAYER

TLS handshake



TLS cipher suite example

- TLS_RSA_WITH_AES_128_CBC_SHA256
  - TLS ⇒ (Lastest version of) TLS
  - RSA ⇒ RSA key exchange
  - WITH ⇒ merely filler
  - AES_128 ⇒ 128-bit AES encryption
  - CBC ⇒ Cipher block chaining
  - SHA256 ⇒ HMAC-SHA256 digest

TLS / SSL key exchange

- Specified by the cipher suite
- Each key exchange algorithm works with some kind of server public key, including
  - RSA
  - DH
  - DH_RSA
  - DHE_RSA

TLS / SSL RSA

- RSA:
  - Server has a certificate with a RSA public key
  - Client creates random “pre-master secret” and encrypts it with the server's public key

TLS / SSL DH

- DH:
  - Server has a certificate with a DH public key
  - Client creates random “pre-master secret” and encrypts it with the server's public key

TLS / SSL DH_RSA

- DH_RSA:
  - Server has a certificate with a DH public key
  - The 'RSA' part means that the DH certifacte has been signed by a CA using RSA keys
  - Client creates random “pre-master secret” and encrypts it with the server's public key

TLS / SSL DHE_RSA

- DHE_RSA:
  - Server has a certificate with a RSA public key but it's only used for signature
  - When a client connects, the server generates a new DH key pair and sends the public key to the client as a ServerKeyExchange message; the server signs that message with its permanent RSA private key
  - The client will respond with a ClientKeyExchange message which also contains a newly-generated DH public key. The DH yields the "pre-master secret".

TLS/SSL versions n

Website protocol support

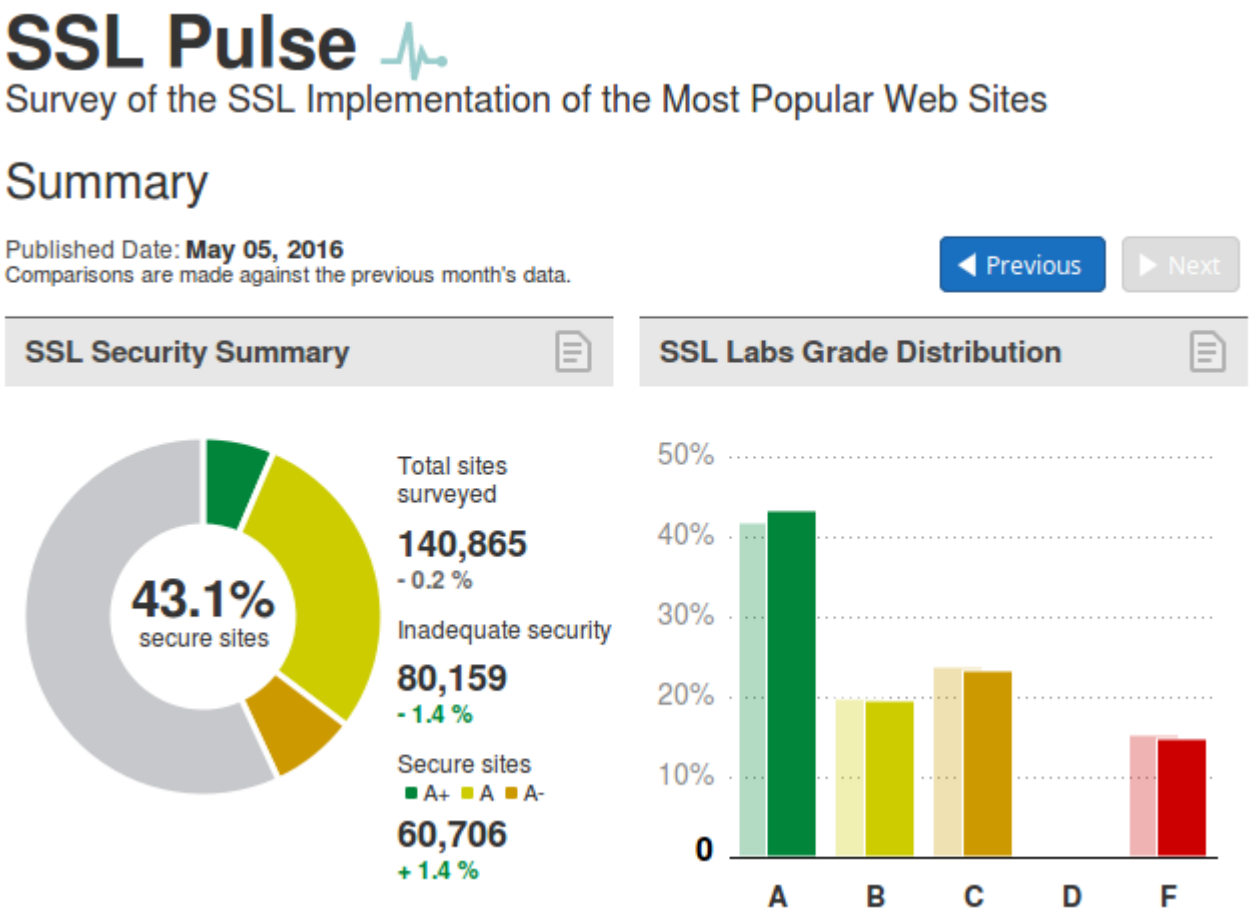
Protocol version	Website support[35]	Security[35][36]
SSL 2.0	7.0% (−0.4%)	Insecure
SSL 3.0	24.4% (−1.0%)	Insecure[37]
TLS 1.0	97.8% (−0.2%)	Depends on cipher[n 1] and client mitigations[n 2]
TLS 1.1	74.0% (+1.0%)	Depends on cipher[n 1] and client mitigations[n 2]
TLS 1.2	76.3% (+0.9%)	Depends on cipher[n 1] and client mitigations[n 2]
TLS 1.3 (Draft)	N/A	

Attacks on TLS/SSL

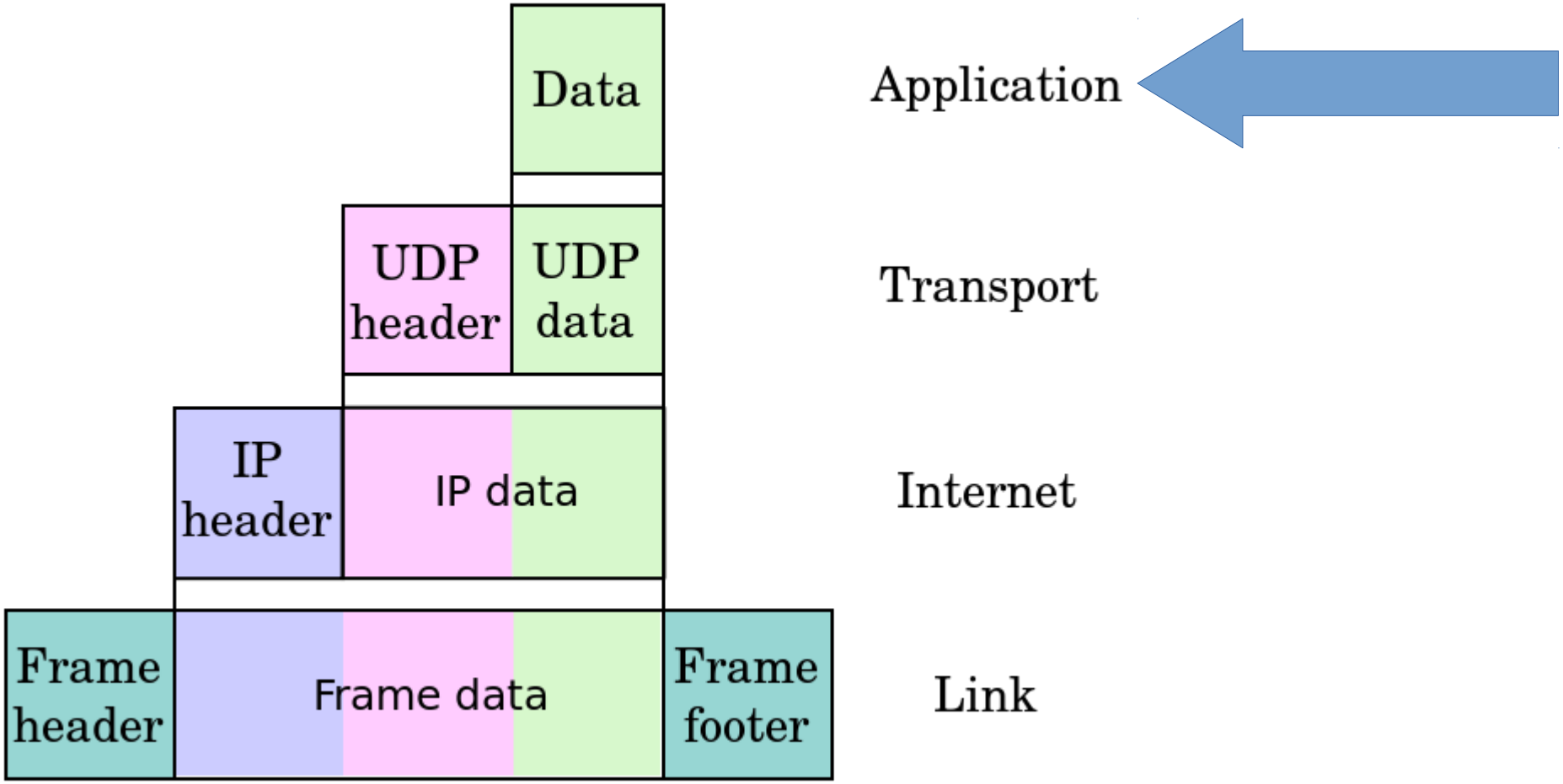
- Renegotiation attack: (2009)
- Timing attacks: Lucky Thirteen
- Padding attack: POODLE
- Downgrade attack: DROWN
- Compression attack: CRIME, BREACH
- Faulty CBC in TLS 1.0: BEAST
- Implementation attack: HEARTBLEED

SSL Pulse

- https://www.trustworthyinternet.org/ssl-pulse/



Where to encrypt?



SSH

The Secure Shell (SSH)

- Protocol for encrypted login and shell connections
- The primary goal is to *improve security* on the Internet. SSH attempts to do this in a way that is easy to deploy, at the cost of absolute security

SSH quotes

- 9.3.1. Confidentiality

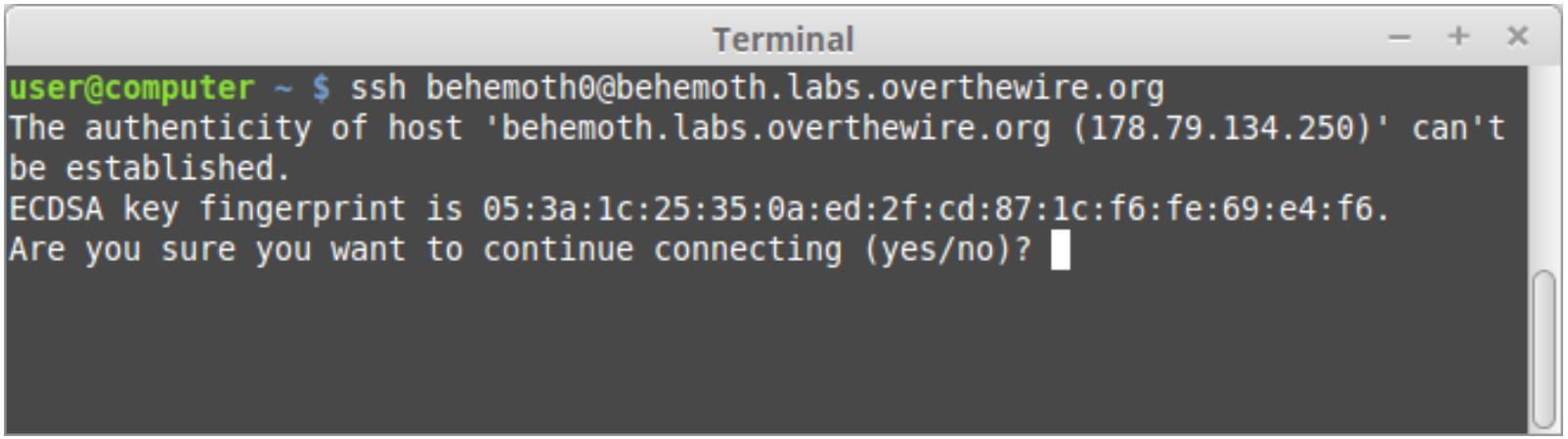
  It is beyond the scope of this document and the Secure Shell Working Group to analyze or recommend specific ciphers other than the ones that have been <u>established and accepted within the industry</u>

SSH Simplified login sequence

- Client contacts server, port 22 standard
- Server sends its public key, and a list of ciphers
- Client authenticates the server
- Client generates a session key, encrypts it with server key
- Server decrypts it and uses it for traffic encryption
- Client authenticates to the server

“Client authenticates the server”

- Server is sending a key, not a certificate

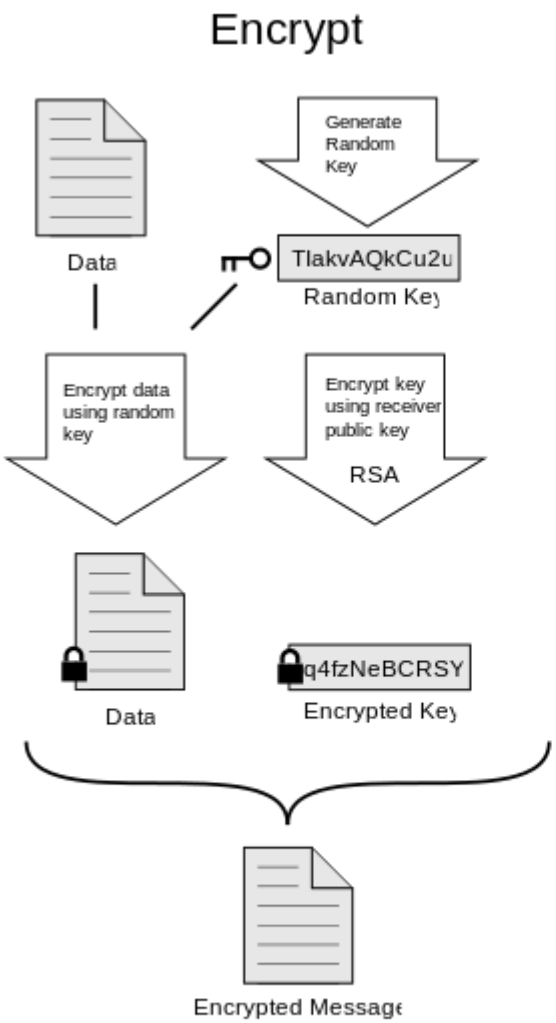
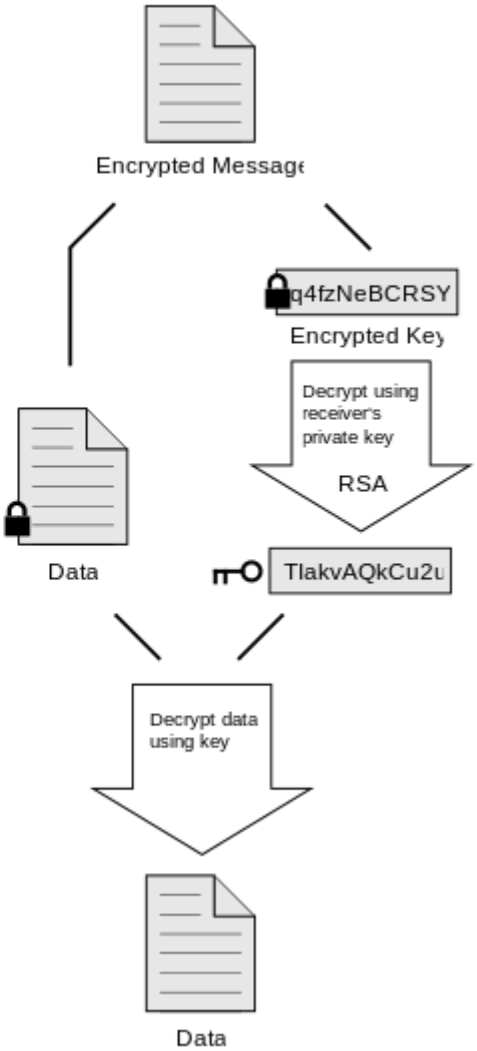


- Client to accept or not to accept
- Pre-distribute known keys? Publish list of signed server keys? Will users care?

“Client authenticates *to* the server”

- Options include:
  - Username/password
  - Client public key authentication

    (Again, a simple key, not certificate)

PGP

File or mail encryption